

Modelado GAM para O3

Equipo 4

2026-09-04

Table of contents

Objetivo de este archivo	1
Carga de la base de modelado	2
Construcción del horizonte	3
Preparación de los predictores del GAM	4
Selección de variables	4
Preparación de la base para $h = 1$	4
Modelo de persistencia (recalculado sobre el subconjunto del GAM)	5
GAM	6
Especificación del modelo	6
Diagnóstico de concurvity	6
Resumen del modelo	7
Visualización de los efectos	9
Modelo alternativo con familia Gamma (link log)	10
Comparación con persistencia en validación 2024	13
Extensión a horizontes de 6, 12 y 24 horas	14
Evaluación	17

Objetivo de este archivo

Igual que el archivo de XGBoost, este parte directamente de `data/base_modelado_o3.rds`.

- estaciones: NTE, NTE2, NO y NE;
- periodo útil: 2021-2025;
- entrenamiento: 2021-2023;
- validación: 2024;
- prueba final: 2025;
- horas objetivo: 10:00-20:00;
- horizonte máximo: 24 horas.

A diferencia de XGBoost, GAM no maneja valores faltantes de forma nativa. Cualquier fila con NA en algún predictor se elimina antes de ajustar el modelo. Esto significa que el conjunto de entrenamiento/validación del GAM es más pequeño que el usado por XGBoost.

El desarrollo se hace de forma “incremental”. Primero se construye el modelo para una hora de anticipación ($h = 1$), y después se extiende la misma lógica a 6, 12 y 24 horas para comparar contra XGBoost en los mismos horizontes.

Carga de la base de modelado

```
library(tidyverse)
library(here)
library(lubridate)
library(mgcv)
library(gratia) # para visualizar los smooths de forma más clara que plot.gam base

options(scipen = 999)
set.seed(123)
```

Primero se lee la misma base de datos utilizada por XGBoost.

```
ruta_base_modelado <- here("data", "base_modelado_o3.rds")

if (!file.exists(ruta_base_modelado)) {
  stop(
    "No se encontró data/base_modelado_o3.rds. ",
    "Renderiza primero cuarto/reporte.qmd para generar la base de modelado."
  )
}

base_origen_modelado <- readRDS(ruta_base_modelado)
```

```

columnas_requeridas <- c(
  "Estacion", "fecha_hora",
  "O3_t", "O3_lag_1", "O3_lag_2", "O3_lag_3", "O3_lag_24",
  "SR", "TEMP", "RH", "WS", "WD_sin", "WD_cos", "NOX"
)

faltantes_estructura <- setdiff(
  columnas_requeridas,
  names(base_origen_modelado)
)

if (length(faltantes_estructura) > 0) {
  stop(
    "La base de modelado no contiene: ",
    paste(faltantes_estructura, collapse = ", ")
  )
}

```

Construcción del horizonte

```

construir_base_horizonte <- function(h) {
  base_origen_modelado |>
    group_by(Estacion) |>
    mutate(
      O3_objetivo = lead(O3_t, h)
    ) |>
    ungroup() |>
    mutate(
      Horizonte_h = h,
      fecha_objetivo = fecha_hora + hours(h),
      Año_objetivo = year(fecha_objetivo),
      Hora_objetivo = hour(fecha_objetivo),
      Mes_objetivo = month(fecha_objetivo),
      Periodo = case_when(
        Año_objetivo %in% 2021:2023 ~ "Entrenamiento",
        Año_objetivo == 2024 ~ "Validacion",
        Año_objetivo == 2025 ~ "Prueba",
        TRUE ~ NA_character_
      )
    )
}

```

```

) |>
  filter(
    !is.na(Periodo),
    between(Hora_objetivo, 10, 20)
  )
}

calcular_metricas <- function(observado, predicho) {
  tibble(
    MAE = mean(abs(observado - predicho)),
    RMSE = sqrt(mean((observado - predicho)^2))
  )
}

```

Preparación de los predictores del GAM

Selección de variables

```

predictores_gam <- c(
  "O3_t", "O3_lag_24",
  "SR", "TEMP", "RH", "WS", "WD_sin", "WD_cos",
  "Hora_objetivo", "Mes_objetivo", "Estacion"
)

```

Hasta este chunk se deja fuera O3_lag_1, O3_lag_2 y O3_lag_3 para evitar concurvity con O3_t.

Preparación de la base para h = 1

```

base_h1 <- construir_base_horizonte(1) |>
  filter(!is.na(O3_objetivo))

resumen_h1_completo <- base_h1 |>
  drop_na(all_of(predictores_gam)) |>
  count(Periodo, name = "Casos_completos") |>
  left_join(
    base_h1 |> count(Periodo, name = "Objetivos_disponibles"),

```

```

    by = "Periodo"
  ) |>
  mutate(Cobertura_pct = round(100 * Casos_completos / Objetivos_disponibles, 1)) |>
  arrange(factor(Periodo, levels = c("Entrenamiento", "Validacion", "Prueba")))

knitr::kable(
  resumen_h1_completo,
  caption = "Casos completos disponibles para el GAM (h = 1)"
)

```

Table 1: Casos completos disponibles para el GAM (h = 1)

Periodo	Casos_completos	Objetivos_disponibles	Cobertura_pct
Entrenamiento	39043	45110	86.6
Validacion	8682	15547	55.8
Prueba	3471	7467	46.5

```

base_h1_completo <- base_h1 |>
  drop_na(all_of(predictores_gam)) |>
  mutate(Estacion = factor(Estacion))

entrenamiento_h1 <- base_h1_completo |> filter(Periodo == "Entrenamiento")
validacion_h1 <- base_h1_completo |> filter(Periodo == "Validacion")

```

Modelo de persistencia (recalculado sobre el subconjunto del GAM)

```

metricas_persistencia_h1_gam <- calcular_metricas(
  validacion_h1$O3_objetivo,
  validacion_h1$O3_t
) |>
  mutate(Modelo = "Persistencia", Observaciones = nrow(validacion_h1), .before = 1)

```

GAM

Especificación del modelo

```
modelo_gam_h1 <- bam(
  O3_objetivo ~
    s(O3_t, k = 10) +
    s(O3_lag_24, k = 10) +
    s(SR, k = 10) +
    s(TEMP, k = 15) +
    s(RH, k = 15) +
    s(Ws, k = 10) +
    WD_sin +
    WD_cos +
    s(Hora_objetivo, k = 8) +
    s(Mes_objetivo, bs = "cc", k = 10) +
    Estacion,
  data = entrenamiento_h1,
  method = "fREML",
  discrete = TRUE
)
```

Diagnóstico de concurvity

A continuación se revisa si algún smooth se puede aproximar a partir de otros.

```
concurvity(modelo_gam_h1, full = TRUE)
```

	para	s(O3_t)	s(O3_lag_24)	s(SR)	s(TEMP)	s(RH)
worst	0.9921954	0.9787671	0.9819640	0.8444530	0.8672480	0.7104020
observed	0.9921954	0.9123019	0.9633558	0.5617978	0.6295574	0.6624482
estimate	0.9921954	0.8822583	0.8649406	0.7522887	0.7722623	0.5846014

	s(Ws)	s(Hora_objetivo)	s(Mes_objetivo)
worst	0.9417892	0.5995789	0.7117088
observed	0.4938760	0.5693350	0.4220718
estimate	0.7418486	0.5750338	0.1752768

Resumen del modelo

```
summary(modelo_gam_h1)
```

Family: gaussian

Link function: identity

Formula:

```
03_objetivo ~ s(03_t, k = 10) + s(03_lag_24, k = 10) + s(SR,
  k = 10) + s(TEMP, k = 15) + s(RH, k = 15) + s(WS, k = 10) +
  WD_sin + WD_cos + s(Hora_objetivo, k = 8) + s(Mes_objetivo,
  bs = "cc", k = 10) + Estacion
```

Parametric coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	68.56371	0.39208	174.873	< 0.0000000000000002 ***
WD_sin	-0.40921	0.08370	-4.889	0.00000102 ***
WD_cos	0.28065	0.06156	4.559	0.00000516 ***
EstacionNTE2	-1.54788	0.15931	-9.716	< 0.0000000000000002 ***
EstacionNO	-0.39008	0.16953	-2.301	0.021403 *
EstacionNE	-0.61939	0.16316	-3.796	0.000147 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Approximate significance of smooth terms:

	edf	Ref.df	F	p-value
s(03_t)	8.575	8.897	8132.47	<0.0000000000000002 ***
s(03_lag_24)	6.744	7.515	38.67	<0.0000000000000002 ***
s(SR)	8.200	8.726	79.67	<0.0000000000000002 ***
s(TEMP)	9.674	11.252	12.10	<0.0000000000000002 ***
s(RH)	6.396	7.924	24.72	<0.0000000000000002 ***
s(WS)	7.790	8.406	204.15	<0.0000000000000002 ***
s(Hora_objetivo)	6.805	6.986	965.61	<0.0000000000000002 ***
s(Mes_objetivo)	7.069	8.000	32.41	<0.0000000000000002 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

R-sq.(adj) = 0.861 Deviance explained = 86.2%

fREML = 1.3104e+05 Scale est. = 47.769 n = 39043

```
gam.check(modelo_gam_h1)
```

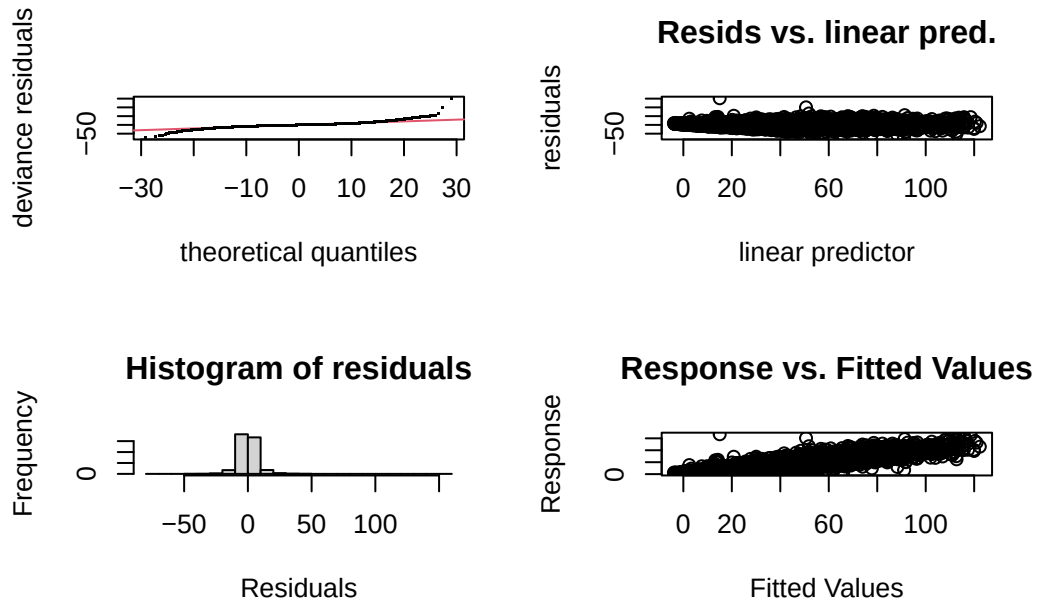


Figure 1: Diagnóstico de residuos del GAM para $h = 1$

```
Method: fREML   Optimizer: perf chol
$grad
[1] -0.0000000000627373709 -0.0000000000146531676  0.0000000000086450846
[4]  0.0000000000264197553  0.0000000000020290436  0.00000000003663203074
[7]  0.0000000000142330592  0.0000000000008344436  0.00000000008149072528

$hess
      [,1]      [,2]      [,3]      [,4]      [,5]
2.9743489389  0.0244622044 -0.0009925257 -0.0180497847 -0.0013688298
0.0244622044  2.1129147488  0.0031934733 -0.0065243883  0.0090824376
-0.0009925257  0.0031934733  2.9821352915  0.0091335780  0.0051303864
-0.0180497847 -0.0065243883  0.0091335780  1.5542562152  0.0008101836
-0.0013688298  0.0090824376  0.0051303864  0.0008101836  1.2596954646
-0.0028651791 -0.0026378145  0.0075759254 -0.0077136894  0.0098574726
-0.0001196131  0.0020518215  0.0173987329 -0.0011651928  0.0028825732
0.0066394725  0.0005286121  0.0216764856  0.0358260338 -0.0198171815
d -3.7873880146 -2.8717900665 -3.5999632932 -4.3367662378 -2.6980428002
```

	[,6]	[,7]	[,8]	[,9]
	-0.002865179	-0.0001196131	0.0066394725	-3.787388
	-0.002637814	0.0020518215	0.0005286121	-2.871790
	0.007575925	0.0173987329	0.0216764856	-3.599963
	-0.007713689	-0.0011651928	0.0358260338	-4.336766
	0.009857473	0.0028825732	-0.0198171815	-2.698043
	3.153608294	0.0047983182	-0.0030983571	-3.395211
	0.004798318	2.8684497147	-0.0011987230	-2.902542
	-0.003098357	-0.0011987230	3.0285219737	-3.534441
d	-3.395211452	-2.9025416057	-3.5344407292	19515.000000

Model rank = 85 / 85

Basis dimension (k) checking results. Low p-value (k-index<1) may indicate that k is too low, especially if edf is close to k'.

	k'	edf	k-index	p-value
s(O3_t)	9.00	8.57	1.00	0.630
s(O3_lag_24)	9.00	6.74	1.00	0.460
s(SR)	9.00	8.20	1.00	0.390
s(TEMP)	14.00	9.67	0.98	0.025 *
s(RH)	14.00	6.40	1.00	0.305
s(WS)	9.00	7.79	0.99	0.200
s(Hora_objetivo)	7.00	6.81	0.99	0.290
s(Mes_objetivo)	8.00	7.07	1.01	0.820

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Visualización de los efectos

```
draw(modelo_gam_h1)
```

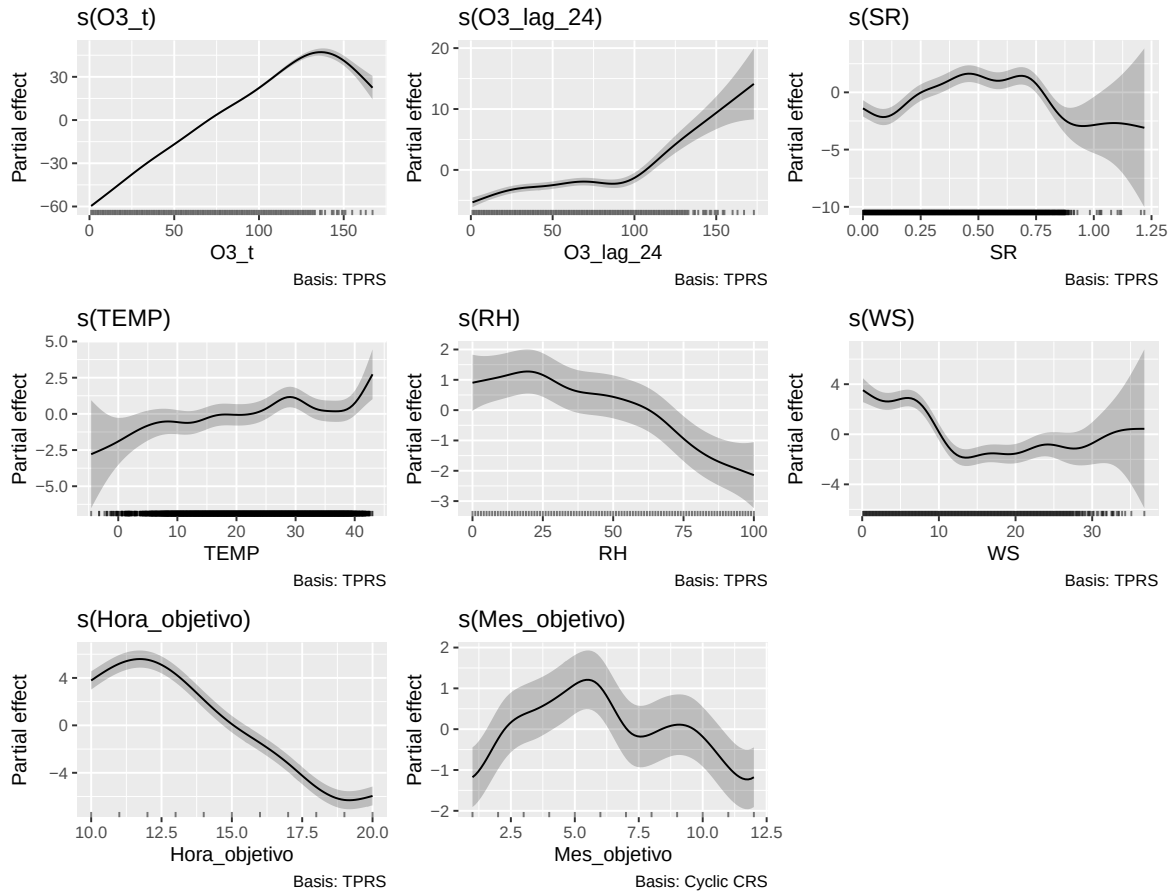


Figure 2: Efecto parcial de cada predictor sobre O3 objetivo ($h = 1$)

Modelo alternativo con familia Gamma (link log)

El diagnóstico de residuos del modelo gaussiano (QQ-plot, residuos vs. predictor lineal, histograma) muestra dos problemas relacionados, los cuales son que la varianza de los residuos crece conforme aumenta el valor predicho (heterocedasticidad) y la cola derecha es más pesada de lo que un modelo gaussiano espera. Esto es consistente con la asimetría de O3 ya documentada en el análisis exploratorio. Como O3 es siempre positivo en la base limpia (mínimo observado = 1, se prueba una familia Gamma con link logarítmico, que asume que la varianza crece con el cuadrado de la media; es un supuesto natural para concentraciones de contaminantes.

```
modelo_gam_h1_gamma <- bam(
  O3_objetivo ~
```

```

s(O3_t, k = 10) +
s(O3_lag_24, k = 10) +
s(SR, k = 10) +
s(TEMP, k = 15) +
s(RH, k = 10) +
s(Ws, k = 10) +
WD_sin +
WD_cos +
s(Hora_objetivo, k = 8) +
s(Mes_objetivo, bs = "cc", k = 10) +
Estacion,
data = entrenamiento_h1,
family = Gamma(link = "log"),
method = "fREML",
discrete = TRUE
)

```

```
gam.check(modelo_gam_h1_gamma)
```

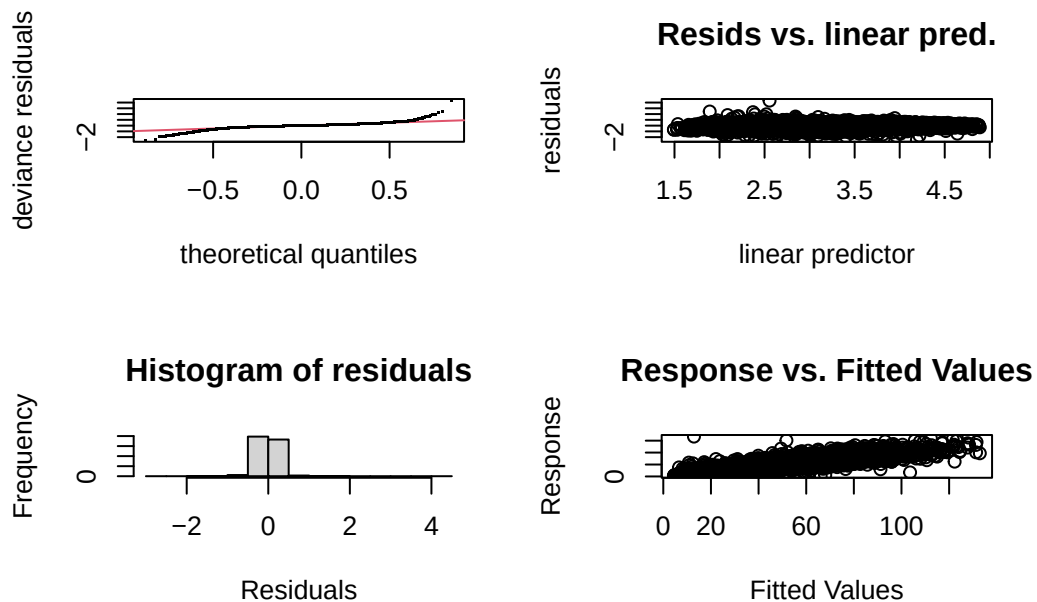


Figure 3: Diagnóstico de residuos del GAM con familia Gamma para $h = 1$

Method: fREML Optimizer: perf chol

\$grad

```
[1] -0.0000012879890 0.0000002270274 0.0000032416385 0.0000021753419
[5] 0.0000007150569 0.0000001564142 -0.0000002528689 0.0000003851668
[9] -0.0034446576647
```

\$hess

```
      [,1]      [,2]      [,3]      [,4]      [,5]
3.703071810 0.029338052 0.002519846 0.004723680 0.0033534305
0.029338052 1.709553727 0.004620336 0.015299524 0.0045130626
0.002519846 0.004620336 1.792299757 0.028387185 0.0033741395
0.004723680 0.015299524 0.028387185 1.695735098 0.0593771263
0.003353430 0.004513063 0.003374139 0.059377126 1.5135066934
0.002922463 0.002124048 0.013011759 -0.003985939 0.0094194577
0.002752419 0.014694082 -0.001691540 -0.005860592 0.0031393639
-0.002773348 -0.006411462 0.014776594 0.001340199 -0.0001432261
d -3.857399171 -2.321953613 -3.109629034 -3.425566804 -2.8770080873
      [,6]      [,7]      [,8]      [,9]
0.002922463 0.002752419 -0.002773348 -3.857399
0.002124048 0.014694082 -0.0064114617 -2.321954
0.013011759 -0.0016915404 0.0147765938 -3.109629
-0.003985939 -0.0058605922 0.0013401988 -3.425567
0.009419458 0.0031393639 -0.0001432261 -2.877008
2.354651540 0.0020529026 0.0035733396 -2.859565
0.002052903 2.7162650644 0.0009582981 -2.879197
0.003573340 0.0009582981 3.1727706300 -3.675419
d -2.859564826 -2.8791965746 -3.6754189706 19515.003445
```

Model rank = 80 / 80

Basis dimension (k) checking results. Low p-value (k-index<1) may indicate that k is too low, especially if edf is close to k'.

	k'	edf	k-index	p-value
s(O3_t)	9.00	8.71	0.95	<0.00000000000000002 ***
s(O3_lag_24)	9.00	5.64	0.98	0.115
s(SR)	9.00	7.22	1.02	0.970
s(TEMP)	14.00	7.85	1.01	0.780
s(RH)	9.00	6.75	0.97	0.015 *
s(WS)	9.00	6.72	0.95	<0.00000000000000002 ***
s(Hora_objetivo)	7.00	6.76	1.00	0.535
s(Mes_objetivo)	8.00	7.35	0.98	0.140

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```
draw(modelo_gam_h1_gamma)
```

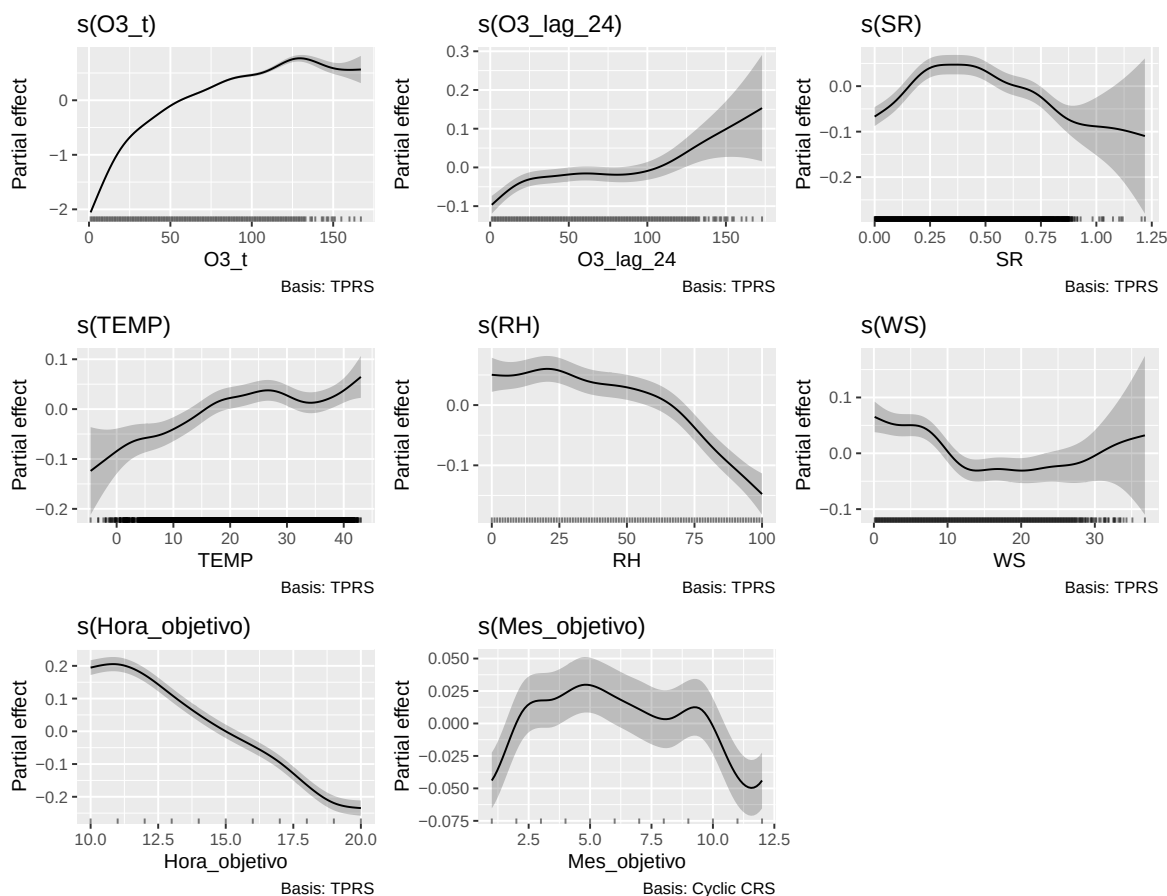


Figure 4: Efecto parcial de cada predictor, familia Gamma ($h = 1$)

Comparación con persistencia en validación 2024

Igual que con XGBoost, la comparación se hace sobre las mismas observaciones. Se incluyen ambos modelos GAM (gaussiano y Gamma) para decidir cuál llevar a los demás horizontes.

```
pred_gam_validacion_h1 <- predict(modelo_gam_h1, newdata = validacion_h1)
pred_gam_gamma_validacion_h1 <- predict(modelo_gam_h1_gamma, newdata = validacion_h1, type =
```

```

metricas_gam_h1 <- calcular_metricas(
  validacion_h1$O3_objetivo,
  pred_gam_validacion_h1
) |>
  mutate(Modelo = "GAM (gaussiano)", Observaciones = nrow(validacion_h1), .before = 1)

metricas_gam_gamma_h1 <- calcular_metricas(
  validacion_h1$O3_objetivo,
  pred_gam_gamma_validacion_h1
) |>
  mutate(Modelo = "GAM (Gamma, link log)", Observaciones = nrow(validacion_h1), .before = 1)

bind_rows(
  metricas_persistencia_h1_gam,
  metricas_gam_h1,
  metricas_gam_gamma_h1
) |>
  mutate(MAE = round(MAE, 3), RMSE = round(RMSE, 3)) |>
  knitr::kable(caption = "Comparación GAM (gaussiano y Gamma) vs. persistencia en validación

```

Table 2: Comparación GAM (gaussiano y Gamma) vs. persistencia en validación 2024 para h = 1

Modelo	Observaciones	MAE	RMSE
Persistencia	8682	7.031	9.590
GAM (gaussiano)	8682	4.884	6.910
GAM (Gamma, link log)	8682	4.933	6.907

Extensión a horizontes de 6, 12 y 24 horas

Se repite la misma lógica para los demás horizontes representativos, manteniendo la misma especificación del modelo. Esto permite comparar directamente contra la tabla de horizontes obtenida con XGBoost.

```

entrenar_y_evaluar_gam_horizonte <- function(h) {
  base_h <- construir_base_horizonte(h) |>
    filter(!is.na(O3_objetivo)) |>
    drop_na(all_of(predictores_gam)) |>
    mutate(Estacion = factor(Estacion))
}

```

```

entrenamiento_h <- base_h |> filter(Periodo == "Entrenamiento")
validacion_h <- base_h |> filter(Periodo == "Validacion")

modelo_h <- bam(
  O3_objetivo ~
    s(O3_t, k = 10) +
    s(O3_lag_24, k = 10) +
    s(SR, k = 10) +
    s(TEMP, k = 15) +
    s(RH, k = 10) +
    s(Ws, k = 10) +
    WD_sin +
    WD_cos +
    s(Hora_objetivo, k = 8) +
    s(Mes_objetivo, bs = "cc", k = 10) +
    Estacion,
  data = entrenamiento_h,
  method = "fREML",
  discrete = TRUE
)

predicciones_h <- predict(modelo_h, newdata = validacion_h)

metricas_persistencia_h <- calcular_metricas(
  validacion_h$O3_objetivo,
  validacion_h$O3_t
)

metricas_gam_h <- calcular_metricas(
  validacion_h$O3_objetivo,
  predicciones_h
)

resumen_h <- tibble(
  Horizonte_h = h,
  Observaciones = nrow(validacion_h),
  MAE_Persistencia = metricas_persistencia_h$MAE,
  MAE_GAM = metricas_gam_h$MAE,
  RMSE_Persistencia = metricas_persistencia_h$RMSE,
  RMSE_GAM = metricas_gam_h$RMSE,
  Mejora_MAE_pct = 100 * (MAE_Persistencia - MAE_GAM) / MAE_Persistencia,
  Mejora_RMSE_pct = 100 * (RMSE_Persistencia - RMSE_GAM) / RMSE_Persistencia
)

```

```

)

list(modelo = modelo_h, resumen = resumen_h)
}

resultado_h1_gam <- tibble(
  Horizonte_h = 1,
  Observaciones = metricas_gam_h1$Observaciones,
  MAE_Persistencia = metricas_persistencia_h1_gam$MAE,
  MAE_GAM = metricas_gam_h1$MAE,
  RMSE_Persistencia = metricas_persistencia_h1_gam$RMSE,
  RMSE_GAM = metricas_gam_h1$RMSE,
  Mejora_MAE_pct = 100 * (MAE_Persistencia - MAE_GAM) / MAE_Persistencia,
  Mejora_RMSE_pct = 100 * (RMSE_Persistencia - RMSE_GAM) / RMSE_Persistencia
)

evaluaciones_horizontes_gam <- map(
  c(6, 12, 24),
  entrenar_y_evaluar_gam_horizonte
)

resultados_horizontes_gam <- bind_rows(
  resultado_h1_gam,
  map_dfr(evaluaciones_horizontes_gam, ~ .x$resumen)
) |>
  arrange(Horizonte_h)

resultados_horizontes_gam |>
  mutate(
    across(c(MAE_Persistencia, MAE_GAM, RMSE_Persistencia, RMSE_GAM), ~ round(.x, 3)),
    Mejora_MAE_pct = round(Mejora_MAE_pct, 1),
    Mejora_RMSE_pct = round(Mejora_RMSE_pct, 1)
  ) |>
  knitr::kable(caption = "Persistencia y GAM en horizontes representativos de 2024")

```

Table 3: Persistencia y GAM en horizontes representativos de 2024

Hori- zonte_h	Observa- ciones	MAE_Per- sistencia	MAE_GAM	RMSE_Per- sistencia	RMSE_GAM	Mejora_MAE_pct	Mejora_RMSE_pct
1	8682	7.031	4.884	9.590	6.910	30.5	27.9
6	8047	23.253	10.727	30.301	14.634	53.9	51.7

Hori- zonte_h	Observa- ciones	MAE_Per- sistencia	MAE_GAM	RMSE_Per- sistencia	RMSE_GAM	Mejora_MAE	Mejora_RMSE_pct
12	7743	23.636	11.375	30.170	15.378	51.9	49.0
24	8571	11.819	10.452	16.354	14.254	11.6	12.8

Con esta tabla se puede comparar el desempeño del GAM contra el XGBoost (MAE_XGBoost y MAE_GAM sobre las mismas métricas, aunque en subconjuntos de datos ligeramente distintos por el tema de casos completos.

Evaluación

El GAM ya se evalúa sobre **2024** en los mismos horizontes representativos (1, 6, 12, 24) que XGBoost. El conjunto de **2025 permanece reservado** para la prueba final y no debe utilizarse para decidir variables, k de los smooths, ni especificación del modelo.